APL Project — Detailed Technical Report

Smart Encryption Advisor & Engine

Table of Contents

1. Executive Summary & Architecture
2. Python Implementation Breakdown
3. Cryptographic Concepts Explained
4. Code Walkthrough
5. Limitations & Edge Cases

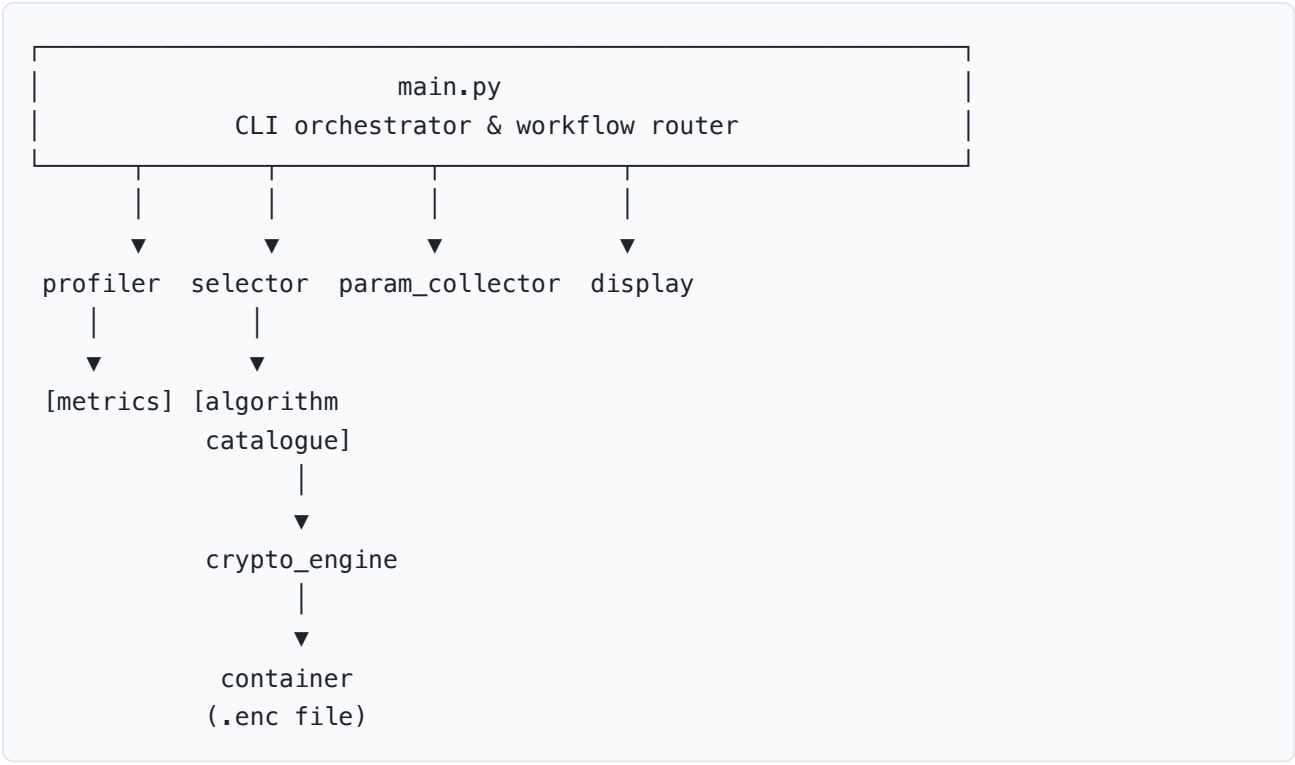
1. Executive Summary & Architecture

1.1 What the Project Does

This project is a **command-line encryption advisor and engine**. It does not blindly encrypt a file with a fixed algorithm. Instead, it first **studies the data** — measuring three scientific metrics — and then **recommends the most appropriate cipher** for that specific data type. The user can accept the recommendation or override it, then the system collects the necessary key material and performs fully authenticated encryption with verified decryption.

The core idea: different data has different characteristics, and the best encryption choice depends on those characteristics. A 10 MB compressed video file is very different from a 5 KB CSV spreadsheet, and they should arguably be encrypted differently.

1.2 Module Architecture



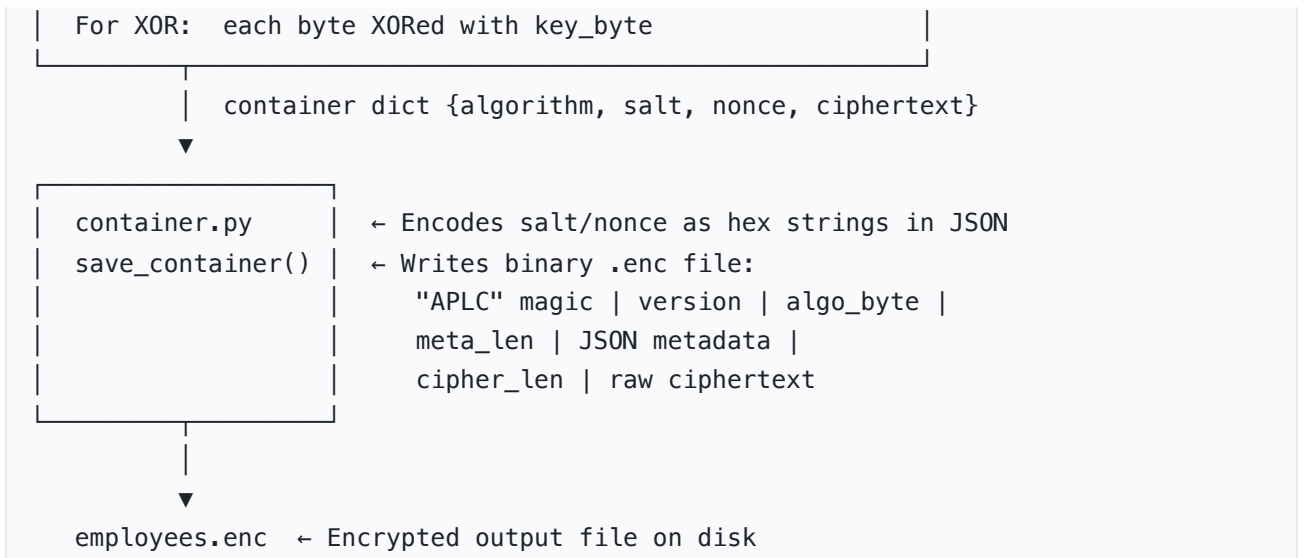
Each module has a **single responsibility**:

Module	Single Responsibility
`profiler.py`	Measure data characteristics
`selector.py`	Map characteristics → algorithm
`param_collector.py`	Securely collect key material from the user
`crypto_engine.py`	Perform all encrypt/decrypt operations
`container.py`	Serialise/deserialise the `.enc` file
`display.py`	Render all terminal output
`main.py`	Wire the modules together into user workflows

1.3 Complete Data Flow

Encryption Path




For XOR: each byte XORed with key_byte
container dict {algorithm, salt, nonce, ciphertext}
container.py
save_container()
← Encodes salt/nonce as hex strings in JSON
← Writes binary .enc file:
"APLC" magic | version | algo_byte |
meta_len | JSON metadata |
cipher_len | raw ciphertext
employees.enc ← Encrypted output file on disk

Decryption Path (Round-trip)



2. Python Implementation Breakdown

2.1 Libraries — What Each One Does and Why

`cryptography` (third-party — PyPI)

What it is: The gold-standard Python cryptography library, maintained by the Python Cryptographic Authority (PyCA). It wraps C-level implementations from OpenSSL (and sometimes

BoringSSL), giving Python access to production-grade, audited, hardware-accelerated cipher implementations.

Why it was necessary: Python's standard library has no implementation of AES-GCM or ChaCha20-Poly1305. Implementing these from scratch would be extremely dangerous — cryptographic correctness requires constant-time comparisons, hardware acceleration, and side-channel resistance that only a mature C library can guarantee.

Specific usage in this project:

```
python
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
from cryptography.exceptions import InvalidTag
```

- `AESGCM` — wraps AES-256-GCM (confidentiality + integrity in one call)
- `ChaCha20Poly1305` — wraps the ChaCha20 stream cipher + Poly1305 MAC
- `InvalidTag` — the specific exception raised when authentication fails; catching it separately from generic `Exception` is important because it tells us *exactly* why decryption failed

The `hazmat` (Hazardous Materials) namespace signals that these are low-level primitives requiring careful, correct usage.

`hashlib` (standard library)

What it is: Python's built-in interface to common hash functions — SHA-256, SHA-512, MD5, and critically for this project, PBKDF2.

Why it was necessary: We need to turn a human-typed passphrase (variable length, not cryptographically uniform) into a fixed-length cryptographic key (32 bytes, statistically uniform). This process is called **Key Derivation** and `hashlib.pbkdf2_hmac` is the function that does it.

Specific usage:

```
python
hashlib.pbkdf2_hmac(
    hash_name  = "sha256",   # underlying PRF
    password   = passphrase.encode("utf-8"),
    salt       = salt,       # 16 random bytes
    iterations = 600_000,    # cost factor
    dklen      = 32,         # output key length
)
```

PBKDF2 applies HMAC-SHA256 **600,000 times** (iterated hashing). This makes brute-force attacks against weak passphrases computationally expensive — each guess requires 600,000 hash operations.

`os` (standard library)

What it is: Python's interface to the operating system. In this project, one specific function is critical.

Why it was necessary: `os.urandom(n)` requests `n` **cryptographically secure random bytes** from the operating system's entropy source (`/dev/urandom` on Linux/macOS, `CryptGenRandom` on Windows). This is essential for two values:

- **Salt —** 16 random bytes, ensures that two users with the same passphrase get completely different derived keys
- **Nonce/IV —** 12 random bytes, ensures that two encryptions of the same file with the same key produce completely different ciphertexts

Python's `random` module uses a pseudo-random number generator (PRNG) and must **never** be used for cryptographic purposes. `os.urandom` is always the correct choice.

`math` (standard library)

What it is: Python's mathematical functions module.

Why it was necessary: Specifically for `math.log2()`, used in the Shannon entropy calculation:

```
python
entropy -= probability * math.log2(probability)
```

Shannon entropy requires logarithm base 2. The result (bits per byte) is the information-theoretic measure of randomness.

`string` (standard library)

What it is: A module containing string constants.

Why it was necessary: `string.printable` is a pre-built string of all 100 printable ASCII characters (letters, digits, punctuation, whitespace). We use it to define the "printable set" for the printable_ratio metric:

```
python
printable_set = set(string.printable.encode("ascii"))
```

Converting it to a `set` makes membership testing O(1) (hash lookup) instead of O(n) (linear scan), which matters when processing large files byte by byte.

`struct` (standard library)

What it is: The module that packs/unpacks primitive C data types (integers, chars) into bytes. This is the foundation of binary file format design.

Why it was necessary: The `.enc` file is a custom binary format. `struct.pack` converts Python integers into big-endian byte sequences for writing to the file. `struct.unpack` does the reverse when reading.

Specific format strings used:

Format	Meaning
`">BB"`	Big-endian, two unsigned chars (1 byte each)
`">H"`	Big-endian unsigned short (2 bytes, max 65535)
`">I"`	Big-endian unsigned int (4 bytes, max 4 GB)

`>` means big-endian (most significant byte first), which is the standard network byte order and ensures the file can be read on any platform regardless of native endianness.

`json` (standard library)

What it is: Python's JSON encoder/decoder.

Why it was necessary: The `.enc` file embeds a JSON block to store human-readable metadata (algorithm name, salt as hex string, nonce as hex string). JSON is self-describing and cross-platform, meaning a file encrypted on macOS can be decrypted on Windows or Linux without special handling.

`pathlib` (standard library)

What it is: An object-oriented filesystem path interface introduced in Python 3.4, replacing string-based `os.path` operations.

Why it was necessary: Used throughout for path manipulation:

- `Path(source).exists()` — check if source is a file
- `path.open("rb")` — open for binary reading
- `out.resolve()` — get the absolute path after writing
- `enc_p.stem` — extract filename without extension (e.g. `"data"` from `"data.enc"`)

`getpass` (standard library)

What it is: A module that prompts for passwords without displaying the typed characters on screen.

Why it was necessary: When the user types their passphrase, `getpass.getpass()` suppresses terminal echo. This is essential because:

1. Shoulder-surfing attackers cannot see the password
2. The passphrase is never written to the terminal scroll buffer
3. Screen recordings or observers cannot capture it

Without getpass: `input("Enter passphrase: ")` would visibly display every character typed.

`argparse` (standard library)

What it is: Python's command-line argument parsing framework.

Why it was necessary: The program supports both an interactive menu and direct one-shot flags:

```
bash
python main.py --encrypt file.csv
python main.py --decrypt file.enc
python main.py --analyse file.csv
```

`argparse.add_mutually_exclusive_group()` ensures these flags cannot be combined (you cannot encrypt and decrypt simultaneously).

2.2 Language Features — Core Python Concepts Used

Type Hints (PEP 484)

Every function signature uses type annotations:

```
python
def shannon_entropy(data: bytes) -> float:
def profile_data(source: str) -> dict:
def encrypt(data: bytes, algo_id: str, params: dict) -> dict:
```

Type hints serve as machine-verifiable documentation. They communicate intent immediately (a reader can tell `shannon_entropy` takes raw bytes and returns a decimal number without reading the body) and tools like `mypy` can statically detect type errors.

Generator Expressions for Byte Processing

Several hot paths use generator expressions to process bytes lazily (one at a time, without building an intermediate list):

```
python
# printable_ratio — counts printable bytes without storing all results
count = sum(1 for b in data if b in printable_set)

# XOR cipher — produces one output byte per input byte on demand
ciphertext = bytes(b ^ key_byte for b in data)

# ROT cipher — applies shift to each byte
ciphertext = bytes(_shift_byte(b, shift) for b in data)
```

The `bytes(generator)` pattern is especially important: it materialises the generator into an immutable `bytes` object. This is memory-efficient for large inputs because at no point is a full intermediate list held in RAM.

Nested Function (Closure) in ROT Cipher

```
python
def _rot_encrypt(data: bytes, shift: int) -> dict:
    def _shift_byte(b: int, s: int) -> int:  # inner function
        if ord('A') <= b <= ord('Z'):
            return (b - ord('A') + s) % 26 + ord('A')
        ...
    ciphertext = bytes(_shift_byte(b, shift) for b in data)
```

`_shift_byte` is defined *inside* `_rot_encrypt`. This keeps the helper logically bound to its parent and invisible to the rest of the module. Python allows functions to be defined anywhere, including

inside other functions.

Dictionary as a Dispatch Table

Instead of a long `if/elif` chain, `selector.py` uses a dictionary to map profiles to algorithms:

```
python
mapping: dict[str, str] = {
    PROFILE_STRUCTURED_TEXT: ALGO_AES_GCM,
    PROFILE_HIGH_ENTROPY   : ALGO_CHACHA20,
    PROFILE_BINARY_STREAM  : ALGO_CHACHA20,
    PROFILE_CLASSICAL      : ALGO_ROT,
}
algo_id = mapping.get(profile, ALGO_AES_GCM)  # safe default
```

This is called the **Strategy Pattern**. Adding a new algorithm only requires adding one dictionary entry, not modifying a conditional chain. The `.get(key, default)` pattern provides a safe fallback without a `KeyError`.

File Handling — Binary Mode vs Text Mode

All file reads and writes use `"rb"` / `"wb"` (raw binary) mode:

```
python
with open(path, "rb") as fh:
    return fh.read()
```

Binary mode is mandatory for cryptographic work because text mode would silently transform bytes — on Windows, `"\n"` becomes `"\r\n"` — which would corrupt ciphertext and break authentication tags. The `with` statement guarantees the file descriptor is closed even if an exception occurs (this is a context manager / `__exit__` mechanism).

Exception Handling — Specific vs Generic

The code practises *specific* exception handling rather than bare `except:` clauses:

```
python
try:
    plaintext = aesgcm.decrypt(nonce, ciphertext, None)
except InvalidTag:               # catch only authentication failure
    raise ValueError("Decryption failed — wrong passphrase…")
```

```
python
try:
    c = enc_container.load_container(enc_path)
except FileNotFoundError as exc:  # file doesn't exist
    display.print_error(str(exc))
except ValueError as exc:          # file is malformed
    display.print_error(f"Invalid container: {exc}")
```

This is deliberate: catching only the expected exception means that unexpected errors (disk errors, memory errors) still propagate as crash-worthy exceptions, preserving the full traceback for debugging.

Module-Level Constants as Configuration

Cryptographic parameters are defined as named constants at the top of `crypto_engine.py`:

```
python
KDF_ITERATIONS  = 600_000
KDF_SALT_BYTES  = 16
AES_KEY_BYTES   = 32
AES_NONCE_BYTES = 12
```

The `_` in `600_000` is Python's numeric literal separator (equivalent to `600000`, purely for readability). Naming constants this way means the iteration count can be updated in one place rather than being buried in function bodies as magic numbers.

`__name__ == "__main__"` Guard

```
python
if __name__ == "__main__":
    main()
```

This standard Python idiom means `main()` is only called when the script is run directly (`python main.py`). When another module imports `main`, this block does not execute. Without it, importing

`main` for testing would immediately launch the interactive menu.

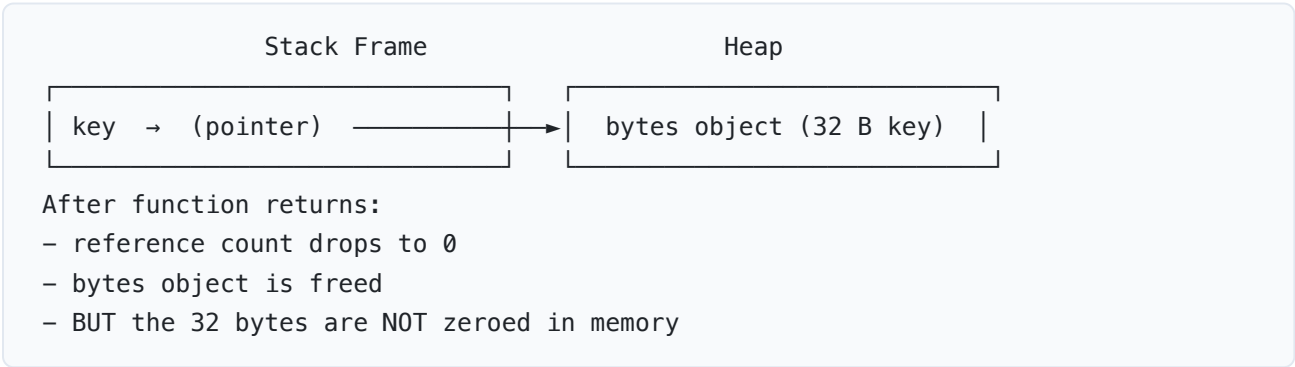
2.3 Memory Management

Python's memory model is relevant to a cryptographic application in several ways:

How Keys Are Stored in CPython

When `_derive_key()` returns a 32-byte key, Python stores it as an immutable `bytes` object on the heap. The variable name (`key`) holds a reference. Python uses **reference counting + cyclic garbage collection (gc module)** for memory management.

Critically: when `key` goes out of scope (e.g. after the encrypt function returns), the reference count drops to zero and CPython's allocator *marks* the memory as reusable — but it does not zero-fill it. This means the key material remains physically in RAM until that memory region is reused for something else.



This is a known limitation of Python for cryptographic use (discussed further in Section 5).

Data Buffering

The full file content is loaded into a single `bytes` object via `fh.read()`. For very large files (>1 GB), this means the entire plaintext exists in RAM simultaneously alongside the ciphertext. This is fine for the scope of this project but would need streaming in production (see Section 5).

The `struct.pack` / `struct.unpack` Pattern

`struct.pack(">H", n)` produces a new 2-byte `bytes` object each time. These are tiny and immediately written to the file buffer, so they don't accumulate. Python's file I/O is internally buffered by the OS, so individual `fh.write()` calls are batched efficiently.

3. Cryptographic Concepts Explained

3.1 Shannon Entropy

Formula: $H = -\sum_{i=0}^{255} p_i \cdot \log_2(p_i)$H = −∑i=0255 pi ⋅ log2(pi) (bits per byte)

Shannon entropy, named after Claude Shannon (father of information theory), measures the **average unpredictability** of a byte in the dataset. The key insight is:

- If every byte in a file is the letter `A` (byte value 65), there is zero surprise — entropy = **0.0 bits/byte**
- If every byte value 0–255 appears with equal probability, the maximum surprise is achieved — entropy = **8.0 bits/byte**
- Plain English text has entropy of about **3.5–5.0 bits/byte** because 'e', 't', 'a' are far more common than 'x', 'q', 'z'
- Already-encrypted or compressed data has entropy approaching **7.5–8.0 bits/byte** because good encryption looks indistinguishable from random noise

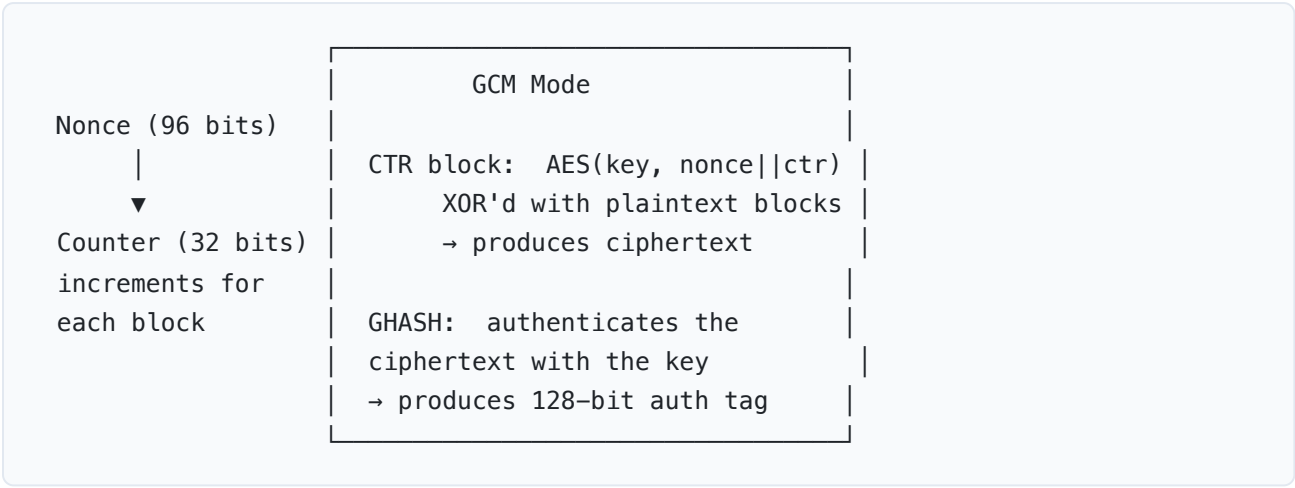
Why this matters for algorithm selection: Data that's already nearly random (entropy > 7.0) is likely already compressed or encrypted. Applying AES-GCM over compressed video doesn't help with compression further; ChaCha20 is preferred because it operates as a stream cipher without block overhead.

3.2 AES-256-GCM

AES (Advanced Encryption Standard) is a symmetric block cipher standardised by NIST in 2001. It operates on 128-bit (16-byte) blocks.

The key size: 256 bits (32 bytes). This means the key space has 2^{256} 2256 possible keys — approximately 10^{77} 1077. Brute-force is physically impossible even with all computational power on Earth.

GCM (Galois/Counter Mode) is the mode of operation — it describes *how* AES is applied to data longer than 16 bytes:



GCM provides two guarantees:

1. **Confidentiality —** the CTR portion XORs plaintext with a keystream, producing ciphertext that is indistinguishable from random
2. **Integrity / Authenticity —** the 16-byte GCM tag is a cryptographic MAC (Message Authentication Code). Any tampering with even one bit of the ciphertext causes `InvalidTag` to be raised on decryption

The Nonce (Number Used Once): A 12-byte (96-bit) random value. The same (key, nonce) pair must NEVER be reused — if it is, the entire security of GCM collapses. This project generates a fresh `os.urandom(12)` nonce for every encryption, so re-use is statistically impossible.

No Padding Needed: Because CTR mode turns AES into a stream cipher (generating a keystream and XORing it with plaintext), there is no concept of block-alignment or padding. The ciphertext is exactly the same length as the plaintext plus the 16-byte authentication tag.

3.3 ChaCha20-Poly1305

Designed by Daniel J. Bernstein (djb). A stream cipher + MAC combination.

ChaCha20 is a stream cipher that generates a keystream from a 256-bit key and 96-bit nonce using a series of quarter-round operations on a 4×4 matrix of 32-bit words. It is entirely built from add-rotate-XOR (ARX) operations, with no S-boxes or lookup tables, making it:

- Immune to cache-timing side-channel attacks
- Extremely fast on processors without AES-NI hardware acceleration (ARM, embedded, older x86)

Poly1305 is a one-time MAC. It computes a 16-byte authentication tag over the ciphertext, providing the same integrity guarantee as GCM's GHASH.

Why recommend it for high-entropy binary? Compressed files, video, images, or already-encrypted blobs have no natural block boundaries. ChaCha20 works cleanly on arbitrary-length byte streams without any padding concern, and on most mobile/ARM processors it is faster than AES.

3.4 PBKDF2-HMAC-SHA256 (Key Derivation)

The user types a passphrase like `"MyDogFido2024!"`. This cannot be used as an AES key directly because:

1. It's not 32 bytes long
2. It's not uniformly random — attackers can guess common passwords

PBKDF2 (Password-Based Key Derivation Function 2) solves this:

```
passphrase + salt  →  HMAC-SHA256  →  (repeat 600,000×)  →  32-byte key
```

Each iteration takes the output of the previous HMAC as the new input. The result after 600,000 iterations is cryptographically indistinguishable from a random 32-byte key — yet fully deterministic given the same passphrase and salt.

The Salt's role: A 16-byte random value stored in the `.enc` file alongside the ciphertext. Two users with the same password `"password123"` get completely different salts and thus completely different derived keys. This defeats pre-computed rainbow table attacks.

Why 600,000 iterations? NIST SP 800-132 (2022 update) recommends a minimum of 600,000 iterations for PBKDF2-HMAC-SHA256 for password-based key derivation. Each decryption attempt costs an attacker 600,000 SHA256 operations — on a modern GPU, this translates to roughly 50,000–100,000 password guesses per second, compared to billions per second for unsalted SHA256.

3.5 Multi-byte XOR (Educational Cipher)

```
Plaintext:   H    e    l    l    o
Byte value:  72   101  108  108  111
Key byte:    42   42   42   42   42   (repeating 0x2A)
XOR result:  98   127  70   70   69  ← ciphertext
```

Why it's insecure: If key_byte = 42 (0x2A), an attacker just needs to try 256 possible key bytes and check if the result looks like valid UTF-8 text. The entire keyspace is $2^8 = 25628$ = 256 —

trivially brute-forceable. With a repeating multi-byte key, frequency analysis on the ciphertext reveals the key length, then each position is independently solvable.

Why it's included: It's a perfect teaching tool for demonstrating XOR's role in stream ciphers, the concept of a repeating keystream, and why short keys are weak.

3.6 ROT Cipher (Caesar Cipher Generalised)

The ROT cipher shifts each letter in the alphabet by a fixed amount:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
                ← shift=13 →
N O P Q R S T U V W X Y Z A B C D E F G H I J K L M
```

Mathematical formula: For uppercase letter with ASCII value `b`: $b' = ((b - 65 + \text{shift}) \mod 26) + 65$ b′ = ((b − 65 + shift) mod 26) + 65

Decryption: Shift in the opposite direction: $\text{reverse} = (26 - \text{shift}) \mod 26$reverse = (26 − shift) mod 26

ROT-13 self-inverse property: With shift=13, applying ROT-13 twice gives back the original (`(13+13) mod 26 = 0`). This is a testable property verified in the test suite.

Why it's insecure: Only 25 possible shifts. An attacker reads the first encrypted word and tries all 25 shifts to find a recognisable English word. Total brute force effort: trivial.

4. Code Walkthrough

4.1 `shannon_entropy()` in `profiler.py`

```
python
def shannon_entropy(data: bytes) -> float:
    if not data:
        return 0.0                    # ① Guard against division by zero

    freq: dict[int, int] = {}
    for byte in data:                 # ② Iterate over bytes (0–255 integers)
        freq[byte] = freq.get(byte, 0) + 1   # ③ Count each byte value

    total   = len(data)
    entropy = 0.0
    for count in freq.values():       # ④ For each unique byte value seen
        probability = count / total   # ⑤ p(x) = occurrences / total
        entropy    -= probability * math.log2(probability)  # ⑥ −p·log₂(p)

    return round(entropy, 4)          # ⑦ Round to 4 decimal places
```

Step-by-step:

1. An empty byte string has no entropy — guard and return 0
2. When iterating a `bytes` object in Python, each element is an `int` (0–255)
3. `freq` builds a frequency map: `{65: 12, 32: 45, 101: 31, ...}` (byte_value → count)
4. We only iterate over byte values that actually appeared; zero-probability terms contribute 0 to entropy anyway (by convention, $0 \cdot \log_2(0) = 0$)
5. Probability of byte value `x` = how many times it appeared / total bytes
6. The Shannon formula: subtract `p·log₂(p)` for each value. The minus sign makes the total positive (since $\log_2$ of a fraction between 0 and 1 is negative)
7. `round(entropy, 4)` prevents floating-point noise like `4.44099999999999`

4.2 `_derive_key()` in `crypto_engine.py`

```
python
def _derive_key(passphrase: str, salt: bytes, key_len: int) -> bytes:
    return hashlib.pbkdf2_hmac(
        hash_name  = "sha256",
        password   = passphrase.encode("utf-8"),   # ① str → bytes
        salt       = salt,
        iterations = KDF_ITERATIONS,               # ② 600,000
        dklen      = key_len,                      # ③ 32 for AES/ChaCha
    )
```

Key points:

1. `passphrase.encode("utf-8")` converts the Python string to raw bytes. This is mandatory because HMAC operates on bytes, not strings. UTF-8 encoding handles international characters correctly (a passphrase could contain é, ü, 日 etc.)
2. `600_000` iterations make this intentionally slow — each call takes ~100–300 ms on a typical machine. That's acceptable for a single legitimate login but catastrophic for an attacker trying millions of guesses per second
3. `dklen=32` makes the output exactly 32 bytes regardless of how long the passphrase was — PBKDF2 stretches or compresses the output to exactly the requested size

Why the same call is used for both encryption and decryption: PBKDF2 is deterministic. Given the same passphrase, salt, and iteration count, it always produces the same bytes. That's the key insight: the salt is stored in the `.enc` file (not secret), and the passphrase is known to the legitimate user. So decryption can always reproduce the exact same key.

4.3 `_aes_gcm_encrypt()` in `crypto_engine.py`

```
python
def _aes_gcm_encrypt(data: bytes, passphrase: str) -> dict:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM  # ①

    salt   = os.urandom(KDF_SALT_BYTES)    # ②  16 random bytes
    nonce  = os.urandom(AES_NONCE_BYTES)   # ③  12 random bytes
    key    = _derive_key(passphrase, salt, AES_KEY_BYTES)  # ④ 32-byte key

    aesgcm     = AESGCM(key)              # ⑤  Create cipher object
    ciphertext = aesgcm.encrypt(nonce, data, None)  # ⑥  Encrypt

    return {                              # ⑦  Return container dict
        "algorithm" : ALGO_AES_GCM,
        "salt"      : salt,
        "nonce"     : nonce,
        "ciphertext": ciphertext,
    }
```

1. The `from ... import` is inside the function body. This is called a *lazy import* — the `cryptography` library is only loaded when this function is actually called. If the user chooses XOR instead, the library is never loaded. Also, it produces a clear ImportError with a helpful install message if the library is missing
2. `os.urandom(16)` — 128 bits of cryptographically secure random salt
3. `os.urandom(12)` — 96 bits of cryptographically secure random nonce. NIST SP 800-38D recommends 96-bit nonces for GCM
4. `_derive_key` runs PBKDF2 — this is the slow step (~100–300ms) and is intentional
5. `AESGCM(key)` initialises the cipher object with the 32-byte key
6. `aesgcm.encrypt(nonce, data, None)` — the `None` is the "additional authenticated data" (AAD) parameter. AAD would allow authenticating header data without encrypting it. We don't use it, so it's `None`. The output includes the plaintext-length ciphertext **plus** a 16-byte GCM authentication tag appended at the end
7. The dictionary return is the "container" pattern — everything needed to decrypt later is packaged together

4.4 `_aes_gcm_decrypt()` in `crypto_engine.py`

```
python
def _aes_gcm_decrypt(container: dict, passphrase: str) -> bytes:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
    from cryptography.exceptions import InvalidTag

    key    = _derive_key(passphrase, container["salt"], AES_KEY_BYTES)  # ①
    aesgcm = AESGCM(key)

    try:
        plaintext = aesgcm.decrypt(container["nonce"], container["ciphertext"],
None)  # ②
    except InvalidTag:        # ③
        raise ValueError(
            "Decryption failed — wrong passphrase or corrupted ciphertext "
            "(GCM authentication tag mismatch)."
        )
    return plaintext
```

1. **Key reproduction:** The salt from the container file is fed back to `_derive_key` with the user-supplied passphrase. If the passphrase is correct, this produces the *exact same 32-byte key* that was used during encryption (PBKDF2 is deterministic)
2. `aesgcm.decrypt()` does two things atomically: verifies the 16-byte GCM tag, then decrypts. It's crucial that tag verification happens *before* returning any plaintext — this prevents partial-decryption attacks
3. `InvalidTag` is caught specifically. This exception is raised when either: (a) the wrong passphrase was used (different key → different tag), or (b) the ciphertext was tampered with in transit/storage. Both cases are detected. The exception is re-raised as a `ValueError` with a user-friendly message

4.5 `save_container()` in `container.py`

```
python
def save_container(container: dict, output_path: str) -> Path:
    algo_byte  = ALGO_TO_BYTE.get(algo, 0xFF)              # ①

    meta_dict  = _container_to_meta(container)             # ②
    meta_bytes = json.dumps(meta_dict, separators=(",",":")).encode("utf-8")  # ③

    ciphertext = container["ciphertext"]

    out = Path(output_path)
    with out.open("wb") as fh:
        fh.write(MAGIC)                                    # ④  "APLC"
        fh.write(struct.pack(">BB", FORMAT_VERSION, algo_byte))  # ⑤
        fh.write(struct.pack(">H", len(meta_bytes)))       # ⑥
        fh.write(meta_bytes)                               # ⑦
        fh.write(struct.pack(">I", len(ciphertext)))       # ⑧
        fh.write(ciphertext)                               # ⑨

    return out.resolve()
```

1. Each algorithm gets a 1-byte identifier (AES-GCM=0x01, ChaCha20=0x02, XOR=0x10, ROT=0x11). This allows future parsers to read the algorithm type before fully parsing the JSON block
2. `_container_to_meta` converts `bytes` fields (salt, nonce) to hex strings for JSON compatibility — `json.dumps` cannot serialise raw `bytes`
3. `separators=(",",":")` produces compact JSON (no spaces), saving a few bytes. `.encode("utf-8")` converts the string to bytes for writing
4. Writing `b"APLC"` as a *magic number* — any file starting with these 4 bytes is recognised as an APL Crypto file. If someone tries to decrypt a random file, the magic check catches it immediately
5. Two 1-byte values packed together: version and algorithm identifier
6. The 2-byte length of the JSON block must be written *before* the JSON data so the reader knows how many bytes to read
7. The actual JSON metadata bytes
8. The 4-byte length of the ciphertext body
9. The raw ciphertext bytes

Why length fields before data? This is the standard TLV (Type-Length-Value) pattern. Without knowing the length ahead of time, a reader cannot know when a variable-length field ends.

4.6 `load_container()` in `container.py`

```
python
def load_container(input_path: str) -> dict:
    with path.open("rb") as fh:
        magic = fh.read(4)
        if magic != MAGIC:                          # ①
            raise ValueError("Not a valid .enc file")

        version, algo_byte = struct.unpack(">BB", fh.read(2))   # ②
        if version != FORMAT_VERSION:
            raise ValueError(f"Unsupported version: 0x{version:02X}")

        algo = BYTE_TO_ALGO.get(algo_byte)          # ③
        if algo is None:
            raise ValueError("Unrecognised algorithm byte")

        meta_len   = struct.unpack(">H", fh.read(2))[0]  # ④
        meta_bytes = fh.read(meta_len)
        meta_dict  = json.loads(meta_bytes.decode("utf-8"))  # ⑤

        cipher_len = struct.unpack(">I", fh.read(4))[0]  # ⑥
        ciphertext = fh.read(cipher_len)

        if len(ciphertext) != cipher_len:           # ⑦
            raise ValueError("File appears truncated")

    return _meta_to_container(meta_dict, ciphertext)  # ⑧
```

1. **Magic validation —** immediately rejects any file that isn't ours. This prevents confusing error messages when a user accidentally passes the wrong file
2. `struct.unpack(">BB", fh.read(2))` reads exactly 2 bytes and unpacks them as two unsigned chars, returned as a tuple `(version, algo_byte)`
3. `BYTE_TO_ALGO` is the *inverse* of `ALGO_TO_BYTE` (built using a dict comprehension: `{v: k for k, v in ...}`). It maps the byte ID back to the string name
4. `[0]` at the end of `struct.unpack(">H", ...)` extracts the first (only) element from the tuple. unpack always returns a tuple even for single values
5. `json.loads` parses the JSON bytes back into a Python dictionary. `.decode("utf-8")` converts bytes to string first
6. Read the ciphertext length, then read exactly that many bytes

7. **Truncation check —** if the file was cut short (disk error, incomplete download), this catches it before passing garbage to the cipher
8. Converts hex strings back to `bytes` objects using `bytes.fromhex()`

4.7 `classify_profile()` in `profiler.py`

```
python
def classify_profile(entropy: float, ratio: float, size_bytes: int) -> str:
    if ratio >= 0.90 and entropy < ENTROPY_LOW_THRESHOLD:   # ①
        return PROFILE_CLASSICAL

    if ratio >= 0.85 and ENTROPY_LOW_THRESHOLD <= entropy <=
ENTROPY_HIGH_THRESHOLD:  # ②
        return PROFILE_STRUCTURED_TEXT

    if entropy > ENTROPY_HIGH_THRESHOLD:   # ③
        return PROFILE_HIGH_ENTROPY

    return PROFILE_BINARY_STREAM           # ④
```

The **decision tree** works in priority order:

1. **Classical check first:** High printability AND low entropy → simple text suitable for ROT/XOR demonstration. If we didn't check this before `structured_text`, plain "AAABBBCCC" text might fall into `structured_text` (high ratio), but its entropy < 3.5 makes it classically weak text
2. **Structured text:** Mostly printable AND moderate entropy → the sweet spot for CSV, JSON, XML, prose. This is the most common case for real-world data files
3. **High entropy:** Entropy above 7.0 regardless of printability. Encrypted, compressed, or binary media. The ratio check is omitted because a JPEG file might have many printable bytes coincidentally
4. **Fallback:** Everything else — binary data that doesn't fit the above patterns — defaults to `binary_stream`

The `size_bytes` parameter is accepted but not currently used in the classification logic (it's reserved for future heuristics, e.g. very small files might warrant different treatment).

5. Limitations & Edge Cases

5.1 Security Weaknesses of This Python Implementation

5.1.1 Key Material in Memory (Most Critical)

The problem: Python's `bytes` and `str` types are **immutable**. When `_derive_key()` returns the 32-byte AES key, it's stored as an immutable `bytes` object. Even after the variable goes out of scope and the garbage collector reclaims the memory, the key bytes remain physically in RAM until that memory page is overwritten by something else. On a system with swap, they could even be written to disk.

Real-world attack: A process with root access or a cold-boot attack could dump physical RAM and find the key material.

Production fix: Use `bytearray` (mutable) instead of `bytes`, and explicitly zero it after use:

```
python
key = bytearray(_derive_key(...))
try:
    # use key
finally:
    for i in range(len(key)):
        key[i] = 0   # overwrite with zeros
```

Or use the `secrets` module's `token_bytes` patterns, or the `cffi`-based `libsodium` wrapper which manages keys in protected memory regions.

5.1.2 No Forward Secrecy

The problem: If the passphrase is compromised today, all past encrypted files can be decrypted retroactively. Every `.enc` file is independently encrypted with a key derived from the same passphrase.

What forward secrecy means: In secure messaging protocols (like Signal), a new ephemeral key is generated for every message. Compromising today's key cannot decrypt yesterday's messages.

Production fix: Implement a ratchet mechanism (e.g. Double Ratchet protocol) where each encryption derives a new key from the previous one via a key-derivation chain.

5.1.3 No Authentication of Metadata in the `.enc` File

The problem: The JSON metadata block (which contains the salt, nonce, and algorithm name) is stored in plaintext and is **not** covered by the GCM authentication tag. An attacker who can modify

the `.enc` file could change the algorithm field from `"AES-256-GCM"` to `"ROT Cipher"` without the program detecting it.

Why it matters: This is a **downgrade attack**. By swapping the algorithm label, an attacker tricks the program into using the weaker cipher for decryption.

Production fix: Include the full metadata in the AEAD "additional authenticated data" (AAD) parameter. Currently we pass `None`:

```
python
ciphertext = aesgcm.encrypt(nonce, data, None)
# Should be:
aad = json.dumps(meta_dict).encode()
ciphertext = aesgcm.encrypt(nonce, data, aad)
```

Any modification to the metadata would then cause `InvalidTag` on decryption.

5.1.4 Nonce Collision Probability

The problem: The nonce is 12 random bytes (96 bits). By the birthday paradox, after 2^{48} 248 encryptions (~281 trillion), there's a 50% chance of a nonce collision with the same key. A nonce collision in AES-GCM completely breaks confidentiality.

Production mitigation: Use a counter-based nonce (deterministic, guaranteed unique) or a 192-bit nonce like XSalsa20. For this project's intended use (personal file encryption), 2^{48} 248 encryptions of the same file with the same key is not a realistic concern.

5.1.5 Timing Side-Channel in Passphrase Comparison

The confirmation step in `_prompt_passphrase`:

```
python
if passphrase != confirm_pp:
```

Python's string comparison is short-circuit — it returns `False` as soon as the first differing character is found. In theory, a very precise timer could measure how many characters matched before the comparison returned False. This is a theoretical concern only in a CLI context (the comparison is user-to-user, not attacker-to-server).

Production fix: Use `hmac.compare_digest(passphrase, confirm_pp)` which performs a constant-time comparison.

5.1.6 Educational Ciphers (XOR and ROT) Are Not Secure

These are **explicitly labelled** as educational in the code and UI. They are included to demonstrate cipher concepts. They should never be used for real data:

- **XOR** with a single repeating byte is cracked by trying 256 key values
- **ROT** with any shift is cracked by trying 25 shift values
- Neither provides authentication — any modification to the ciphertext produces garbled output without detection

5.1.7 File-at-a-Time Processing (No Streaming)

```
python
with open(path, "rb") as fh:
    return fh.read()   # loads entire file into RAM
```

For a 1 GB video file, this loads 1 GB into RAM, plus the 1 GB ciphertext, for a peak usage of ~2 GB. Python's garbage collector would eventually free the plaintext bytes, but there's a window where both exist.

Production fix: Use chunked streaming with `fh.read(65536)` in a loop, updating the AEAD incrementally. Note: GCM's streaming authentication is more complex — the tag is only finalised at the end.

5.2 Changes Required for Real-World Production Deployment

Authentication & Multi-User Support

Replace single-passphrase key derivation with **public-key cryptography**:

- Generate an RSA or EC key pair per user
- Encrypt the file's symmetric key with the recipient's public key
- Store the encrypted symmetric key in the `.enc` container
- The recipient uses their private key to unwrap the symmetric key

This enables encryption for multiple recipients without sharing a passphrase.

Key Management System (KMS)

Instead of deriving keys from user passphrases, integrate with:

- **AWS KMS**, **Google Cloud KMS**, or **HashiCorp Vault**

- The symmetric key is generated randomly and stored encrypted in the KMS
- The user authenticates to the KMS service (using IAM, OAuth, certificates) to get the key
- Keys have automatic rotation schedules

Hardware Security Module (HSM) Integration

For enterprise use, private keys and encryption keys should live in an HSM (hardware security module) — a tamper-resistant device that performs cryptographic operations without ever exposing the raw key material to software.

Argon2id Instead of PBKDF2

For password-based use cases, replace PBKDF2-HMAC-SHA256 with **Argon2id** (winner of the Password Hashing Competition, 2015). Argon2id is memory-hard — it requires a configurable amount of RAM per guess, which makes GPU-based brute-force much more expensive than PBKDF2:

```
python
# Production-grade alternative
from argon2.low_level import hash_secret_raw, Type
key = hash_secret_raw(
    secret=passphrase.encode(),
    salt=salt,
    time_cost=3,         # iterations
    memory_cost=65536,   # 64 MB per guess
    parallelism=4,
    hash_len=32,
    type=Type.ID,
)
```

Logging & Audit Trail

Real deployments need structured audit logs:

- Who encrypted what file, when
- Who decrypted what file, when (and from which IP)
- Failed decryption attempts (wrong passphrase → possible brute-force)

Input Validation & Sandboxing

- Limit file size before loading into RAM
- Run in a process with restricted filesystem permissions
- Validate that file paths don't escape the intended directory (path traversal protection)

Key Deletion & Secure Wipe

As noted in §5.1.1, implement secure memory zeroing of all key material after use, using `ctypes` or `cffi` to directly zero the underlying C buffer.

5.3 Summary Table

Weakness	Severity	Affects Production?	Fix
Key in Python heap memory	High	Yes	`bytearray` + explicit zeroing
No forward secrecy	Medium	Yes	Ratchet / ephemeral keys
Metadata not authenticated	Medium	Yes	Pass metadata as AEAD AAD
No streaming for large files	Medium	Yes	Chunked I/O
PBKDF2 vs Argon2id	Low-Medium	Yes	Switch to Argon2id
Educational ciphers (XOR/ROT)	High (if misused)	N/A	Already labelled; remove in prod
Nonce collision (theoretical)	Very Low	No (at this scale)	Counter nonce in multi-encrypt
Non-constant-time comparison	Very Low	No (CLI context)	`hmac.compare_digest`

Report generated for APL Project — Smart Encryption Advisor & Engine. All code references point to the implementation in `/Users/tanishkyadav/Desktop/APL Project/`.